IE 643: Deep Learning - Theory and Practice **July-Dec 2026**

Project Report: PINN-based Inversion of Spatially-Varying Diffusivity

Team Name: Binary_Brains *Team Members: 24b2451, 24b2443*

Jash Taparia (24b2451) · Mihir Gune (24b2443)

Code repository: `https://github.com/bloodrepaer/IE643-Binary-Brains-PINN-Inversion` [13]

Abstract

We recover an unknown, spatially varying diffusivity $D(x, y)$ in the two-dimensional heat equation $u_t = \nabla \cdot (D\nabla u)$ from sparse, noisy temperature sensors, and compare a physics-informed neural network (PINN) with classical PDE-constrained optimisation solved by the adjoint method. All data are synthetic. A hand-built finite-volume forward solver (5-point flux form, RK4, Crank–Nicolson and forward Euler, measured orders 4.03 / 2.01 / 1.00) generates 389 benchmark datasets from six families of ground-truth D, with controlled sensor count, layout, timing, horizon and noise. The classical baseline uses an exact discrete adjoint of Crank–Nicolson with L-BFGS-B and a discrepancy-principle regularisation weight, and was run on all 389 datasets. The PINN uses Fourier-feature networks for temperature and $\log D$ with a five-term physics-informed loss. When the initial and boundary conditions are known exactly and there are up to a few hundred sensors, the adjoint is more accurate: it wins 20/20 representative datasets (median relL2(D) 0.23 vs 0.47) and 41/44 datasets in a sensor × noise × layout sweep. The PINN wins in three regimes that we identified and tested with matched controls: (i) an unknown boundary condition or heat source, where the standard adjoint must assume one and fails (PINN better on 21 of 22 such datasets, median relL2(D) 0.38 vs 5.0); (ii) very dense sensor data, where a neural PINN reaches 8.6% coefficient error at 7,500 sensors and 5% noise against 25.1% for a matched neural adjoint; (iii) too few sensors, where it degrades gracefully to the best constant instead of inventing structure. A Streamlit interface lets a user generate, select or upload sensor data, run both methods, and see the metrics and recovered maps.

1 Introduction

Many engineering and biomedical problems need a material property that cannot be measured directly: the thermal conductivity map of a composite plate with defects, the permeability of tissue, or the diffusivity of a drug through a hydrogel. What can be measured is the process that the property governs, here a temperature (or concentration) field $u(x, y, t)$, at a handful of sensor locations and times. Recovering the property from such data is an *inverse problem*. Unlike the forward problem, it is ill-posed: many coefficient fields explain the same sparse readings, and small noise can produce large errors in the estimate.

Two families of methods address it. *Classical PDE-constrained optimisation* represents the unknown with a finite set of parameters, solves the PDE exactly with a numerical solver at every iteration, and obtains the gradient of the data misfit from an adjoint solve [15, 6]. *Physics-informed neural networks* [8] represent the solution, and here also the coefficient, by neural networks, and encourage the PDE through a residual penalty computed by automatic differentiation. PINNs need no mesh, no adjoint derivation and no exact boundary data, which makes them attractive for messy real problems; whether they are also more accurate is an empirical question. The course project asks us to build both methods on the same synthetic data and to find where the PINN is better.

The rest of the report is organised as follows. Section 2 gives the project workflow. We survey the literature in Section 3. Our approaches, split into work before and after the prep-presentation review, are in Section 4, and the datasets in Section 5. Experiments and results are in Sections 6 and 7. The plan for novelty assessment is in Section 8, and Section 9 concludes.

2 Project Workflow

- **Problem definition.** Fixed the PDE (divergence form), domain, initial and boundary conditions, unknown ($\log D$), metrics and the six ground-truth families.
- **Literature review.** PINNs for inverse problems [8], PINNs for spatially varying diffusivity [14], adjoint vs PINN comparisons [17], spectral bias and Fourier features [7, 12].
- **Forward solver and dataset (Deliverable 1).** Finite-volume solver with three time integrators, verified for order of accuracy; 389-dataset benchmark with nested sensors and controlled noise.
- **Classical baseline (Deliverable 3).** Discrete adjoint of Crank–Nicolson, gradient verified by Taylor tests; run on all 389 datasets.
- **PINN (Deliverable 2).** Two-network PINN, loss-weight sweep, 10-case benchmark.
- **Head-to-head experiments (Deliverables 5, 6).** 20-dataset study, 44-dataset sensor/noise/layout sweep, ablations, a solver-bias test, and re-runs of two reference papers.
- **Where should a PINN win? (Deliverable 4).** Hypothesis suites H1–H9 with matched controls (sparse sensors, unknown BC, unknown source, heavy noise, long horizons, dense data, fine features); neural-coefficient campaign where both methods share the same D network; dense-sensor sweep up to 10,000 sensors.
- **Interface and packaging.** Streamlit app with upload, generation, both solvers, metrics and model reload; documented code and trained models.

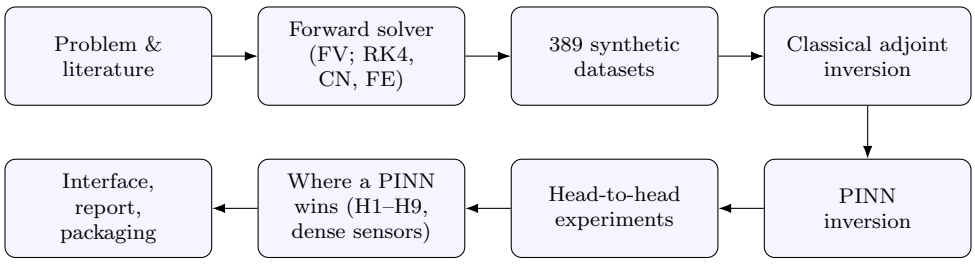


Figure 1: Project workflow from problem statement to deliverables.

3 Literature Survey

Physics-informed neural networks. Raissi, Perdikaris and Karniadakis [8] train a network $u_\theta(x,t)$ on a loss that adds the data misfit to the mean squared PDE residual at collocation points, computed by automatic differentiation. Unknown PDE parameters are trained jointly with θ, so the same framework

solves forward and inverse problems. Two known difficulties shaped our design: networks learn low frequencies first (spectral bias, [7]), which random Fourier feature embeddings mitigate [12]; and the gradients of the different loss terms can be badly unbalanced, so loss weights matter a great deal [16].

PINNs for spatially varying diffusivity. Thakur et al. [14] is the closest published work. They use two networks, one for concentration and one for $D(x, y)$, train on concentration fields of four test cases, use the divergence form $\nabla \cdot (D\nabla c)$, and also propose a backward-Euler consistency loss between consecutive snapshots. They report 4.6–6.3% coefficient error on dense data and 6.45% with 250 points. This motivated our two-network PINN and was the claim we set out to test.

Adjoint versus PINN. Zhang, Alla and Karniadakis [17] compare the two paradigms on Burgers, Darcy and Allen–Cahn problems while holding physics and data fixed. Their conclusion is that the adjoint is preferable when a reliable discrete solver exists and the unknown is low-dimensional, while PINNs become competitive when the unknown is itself a neural field, when the problem is high-dimensional (adjoint storage and solve costs grow quickly), or when the data are very noisy. They also recommend a PINN warm start for the adjoint.

Classical inverse problems. Our baseline follows standard reduced-space optimisation [15]: Tikhonov or total-variation [10] regularisation, gradients from the adjoint-state method [6], bound-constrained L-BFGS-B [1], and the regularisation weight chosen by Morozov's discrepancy principle [4]. The Crank–Nicolson time stepping uses a Rannacher start [9] to damp stiff modes from a non-smooth initial state.

4 Proposed Approach or Approaches

Problem. On $\Omega = [0, 1]^2$ and $t \in [0, T]$,

$$\frac{\partial u}{\partial t} = \nabla \cdot \big(D(x, y)\nabla u\big) = D\,\Delta u + \nabla D \cdot \nabla u, \qquad u(\cdot, 0) = u_0, \quad u|_{\partial\Omega} = g(t). \tag{1}$$

Given noisy readings $d_i = u(x_i, y_i, t_i) + \varepsilon_i$, $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$, we estimate $D > 0$. We always work with $m = \log D$, which keeps D positive and treats relative errors symmetrically. We keep the divergence form: the $\nabla D \cdot \nabla u$ term carries the information about where D changes.

4.1 Work done before prep-presentation review

- Problem formulation, literature review and the plan: forward solver, synthetic dataset, classical adjoint baseline, PINN, and a hybrid PINN→adjoint method.
- First forward solver: Crank–Nicolson with a 9-point stencil, data simulated on the same grid used for inversion.
- First dataset version: absolute noise $\sigma \in \{0.01, 0.05\}$, eight random observation times per file, seeds from Python's `hash()`.

The review exposed weaknesses that we fixed afterwards (Table 1).

Prep presentation	Final	Reason
9-point stencil	5-point flux form, harmonic-mean links	1.3–1.8× more accurate on every family; simpler exact adjoint
Crank–Nicolson only	RK4, Crank–Nicolson, forward Euler behind one interface	RK4 generates data, CN is the inversion solver, FE is a benchmark
Data on the inversion grid	Data on 129^2, inversion on 65^2	Avoids the "inverse crime"
Absolute noise 0.01 / 0.05	Relative noise $\eta \in \{0, 1, 5, 10\%\}$	Absolute σ was 30–280% of the signal
8 random times per file	Fixed time protocols + temporal-density sweep	Reproducible; time becomes a controlled axis
`hash()` seeds	`zlib.crc32` seeds + manifest	`hash()` changes every Python run
Hybrid PINN→adjoint planned	Dropped	Scope; compute spent on more experiments

Table 1: Changes made after the prep-presentation review.

4.2 Work done after prep-presentation review

4.2.1 Forward solver

Space is discretised with node-centred finite volumes on an $N \times N$ grid. Each link between neighbouring nodes p, q carries the harmonic mean $2D_pD_q/(D_p + D_q)$, because the two half-cells act as thermal resistances in series. Eliminating the Dirichlet boundary nodes gives the semi-discrete system $\dot{\mathbf{u}} = L(D)\mathbf{u}+\mathbf{b}(t)$ with L sparse, symmetric and negative definite. Three integrators share this operator: RK4 (data generation), Crank–Nicolson with two backward-Euler half-steps at the start (Rannacher start; used for inversion and replay; unconditionally stable) and forward Euler (benchmark only). The Crank–Nicolson matrix is factorised once per D and reused for every forward and adjoint step.

4.2.2 Classical inversion: discrete adjoint

The unknown is $m = P\theta$, where θ lives on a 16×16 control grid and P is bilinear interpolation to the 65×65 inversion grid; $D = e^m$ with bounds $10^{-3} \le D \le 10$. We minimise

$$J(\theta) = \frac{1}{2\sigma^2}\left\|Q\,\mathbf{u}(m) - \mathbf{d}\right\|^2 + \alpha R(m), \qquad R_{\mathrm{Tik}} = \tfrac{1}{2}\int |\nabla m|^2, \quad R_{\mathrm{TV}} = \int \sqrt{|\nabla m|^2 + \epsilon}, \tag{2}$$

where $\mathbf{u}(m)$ is the Crank–Nicolson solution (a hard constraint) and Q interpolates bilinearly in space and linearly in time, so sensors and times may be off-grid. The gradient with respect to all 256 parameters comes from one forward and one backward (adjoint) sweep that mirrors the time stepping exactly, including the Rannacher steps. We optimise with L-BFGS-B ($\le$ 150 iterations per α), starting from the best constant D, over a decreasing continuation $\alpha_k = N_d\, 10^{-k/2}$ with warm starts. α is chosen by the discrepancy principle: the first α whose RMS misfit is at most σ.

4.2.3 PINN

Architecture. The temperature network maps (x, y, t) through random Fourier features (scale 0.35) to three hidden `tanh` layers of 80 units (19,521 parameters). Its output is an analytic heat-kernel approximation of the initial bumps plus a learned correction multiplied by $16x(1 - x)y(1 - y)$ and $\tanh(8t/T)$, so the zero boundary and the initial condition hold by construction. The coefficient network maps (x, y) through Fourier features to three `tanh` layers of 64 units (12,545 parameters) and outputs a correction to a trainable background $\log D$, tapered to the background near the walls, with $D = \exp(\mathrm{clamp}(m, -5, 2))$.

We use `tanh` because the loss needs second derivatives, and ReLU networks have zero curvature almost everywhere.

Loss.

$$\mathcal{L} = 40\,\mathcal{L}_{\text{data}} + 20\,\mathcal{L}_{\text{PDE}} + 10\,\mathcal{L}_{\text{BC}} + 20\,\mathcal{L}_{\text{IC}} + \lambda\,R(\log D). \tag{3}$$

$\mathcal{L}_{\text{data}}$ is the mean squared misfit normalised by the mean squared reading, damped once it falls within $0.95\sigma^2$ so the network does not chase the noise. $\mathcal{L}_{\text{PDE}}$ is the mean squared residual of $u_t - (D_x u_x + D_y u_y + D\Delta u)$ at 768 collocation points per step. 90% of its gradient is routed to the D network and 10% to the u network, so the PDE mostly shapes D rather than distorting u. R is Tikhonov ($\lambda = 2 \times 10^{-4}$, smooth families) or TV (3×10^{-4}, sharp families).

Training. AdamW [2, 3] (weight decay 10^{-5}), learning rates 1.5×10^{-3} (u) and 3.5×10^{-3} (D) with cosine annealing to 10^{-5}, gradient clipping at 1.0, 1,500 epochs. The D network's rate is held near zero for about 50 epochs and ramped over 15% of training, so u fits the data first. We keep the best checkpoint from the last 35% of training. Implemented in PyTorch [5].

Boundary-aware variant. To test unknown boundary conditions, `pinn_inversion_bc.py` reads the boundary mode from the problem file. With a known non-zero boundary it replaces the hard zero window by a soft boundary loss. With an unknown boundary (`free`) it has no window and no boundary loss, so wall values are determined only by the data and the PDE.

4.2.4 Neural-coefficient extension

The grid adjoint and the PINN differ in two ways at once: how D is represented (16×16 grid vs network) and how the physics is enforced (exact solver vs penalty). To separate them, we built a shared coefficient network (`NeuralDiffusivity`), a float64 `tanh` MLP with 3 layers of 32 units whose output $z_\phi(x, y)$ is mapped to a bounded coefficient,

$$\log D = \log D_{\min} + \log(D_{\max}/D_{\min})\,\text{sigmoid}(z_\phi), \qquad D \in [10^{-3}, 10]. \tag{4}$$

Its last layer is initialised to reproduce the best constant fitted to the data. Both methods use it with the same seed:

- **Neural adjoint**: the same Crank–Nicolson solver and discrete adjoint give $\partial J/\partial m$ on the grid, which is back-propagated through the network (a vector–Jacobian product); optimised with L-BFGS-B.
- **Neural PINN**: a temperature MLP trained jointly with the same D network on data and PDE residual (Adam followed by L-BFGS).

For the dense-sensor study we used matched temporal variants. The PINN's state network has hard initial and boundary conditions and a backward-Euler consistency defect on 4,096 random time pairs ($\Delta t/T = 0.01$). The adjoint uses backward Euler with 100 full steps. Both use $\alpha = 1$ and seed 42, and run 1,000 Adam steps followed by 10×400 self-scaled Broyden (SSBroyden) quasi-Newton iterations, as in [17].

4.2.5 Interface

The interface is a Streamlit [11] web app (`app/streamlit_app.py`) over a thin backend (`app/core.py`) that calls the project's solvers unchanged. It has four tabs.

1. **Data.** Three ways to get sensor data: generate a synthetic dataset (any of 17 D fields or an uploaded D map; horizon; 5–100 sensors or the full field; layout; time protocol; noise; boundary condition; seed); pick one of the 389 benchmark datasets; or upload a CSV of readings `x,y,t,u[,sigma]`, optionally with an initial condition and a true D for scoring.

2. **Run.** Classical settings (control grid 8–32, Tikhonov/TV, iterations) and PINN settings (100–6,000 epochs, regulariser, weight, seed).

3. **Results.** The displayed metrics: replay RMS/σ for both methods, plus relL2(D) and the best-constant baseline when the truth is known. Also maps on a shared colour scale, $\log(\hat{D}/D)$ error maps, profiles, the α-selection curve, a sensitivity map, PINN loss curves, and downloads (trained models, $\hat{D}$ as CSV/NPZ, metrics, figures).

4. **Load model.** Reload a saved `classical_model.npz` or `pinn_best.pt`, or one of 40 bundled trained models. The app rebuilds D from the model file and scores it.

Design choices: every data source is written to the project's own dataset format, so both solvers read user data exactly as they read the benchmark. Generation reuses the benchmark generator, so interface data are statistically identical to the benchmark. Uploads are validated (columns; $x, y \in [0, 1]$; $t > 0$). Both estimates are scored by replaying $\hat{D}$ through the same Crank–Nicolson solver. Results stay in session state, so switching tabs never retrains. Every path was tested headlessly with `streamlit.testing` and in a browser (Figure 9).

5 Data set Details

All data are synthetic and generated by our own code, so no data were downloaded from external websites. One command, `python generate_dataset_v2.py` (about 4 minutes on a laptop CPU), writes the benchmark `datasets_v2_rk4_zero/`: 389 datasets and 35 ground-truth simulations, 175 MB.

Ground-truth fields. Six families plus 11 variants: A smooth sinusoid; B Gaussian bump; C circular inclusion (sharp edge; high/low-contrast variants); D layered (and coarse/fine checkerboards); E Gaussian random field (short/long correlation variants); F plate with defects (single, many, small, voids, low contrast).

Generation steps.

1. Build the true D on the 129^2 data grid; the inversion-grid truth is every second node.

2. For each horizon (Fourier number) Fo $\in \{0.02, 0.05, 0.1, 0.2\}$ set $T = \text{Fo}/\bar{D}$. The initial condition is two Gaussian bumps; the boundary is held at zero (other scenarios: heated, ramped or pulsed edge).

3. Simulate u with RK4 on 129^2 at its largest stable step; store 401 output times and 41 snapshots.

4. Fix one noise level per (family, Fo, η): $\sigma = \eta\times$ RMS of a reference signal.

5. Choose sensors from a fixed pool of 100 per (family, layout), so sets are nested ($5 \subset 20 \subset 50 \subset 100$). Layouts are random, grid, clustered and boundary-only.

6. Choose observation times from a protocol: `uniform4`…`uniform200`, `log8`, `early8`, `late8`, `final1`.

7. Interpolate the clean field at each (sensor, time) and add Gaussian noise with seed `crc32(name)`.

Sweeps. S1 sensors $\{5, 20, 50, 100, \text{full}\}\times$ noise $\{0, 1, 5, 10\%\}$; S2 horizon $\times$ time protocol; S3 time density $\times$ noise; S4 layout $\times\{20, 50\}$ sensors. That is 63 datasets per main family $\times$ 6, plus 11 variant baselines, for 389 in total. Dataset keys read `<family>__Fo<Fo>__<protocol>__n<count>__<layout>__eta<η>`.

Format. `problem/` holds the PDE, T, the initial condition and the boundary values. `sensors/<key>.csv` has columns `x,y,t,u,sigma`, one row per reading (also the interface upload format). `ground_truth/` holds the true D and snapshots, used only for scoring. A manifest records every configuration and seed. No further pre-processing is applied. The methods read the readings directly, and the PINN normalises time by T.

Additional collections (same format, generated by scripts in the package): the hypothesis suites H1–H5 (57 datasets) and H6/H7/H9 (24), each with matched controls; a nested density extension (180 configurations, 5–3,969 sensors on the inversion grid, 1/5/10% noise); and dense sets of 1,000–10,000 sensors sampled from the 129^2 simulation.

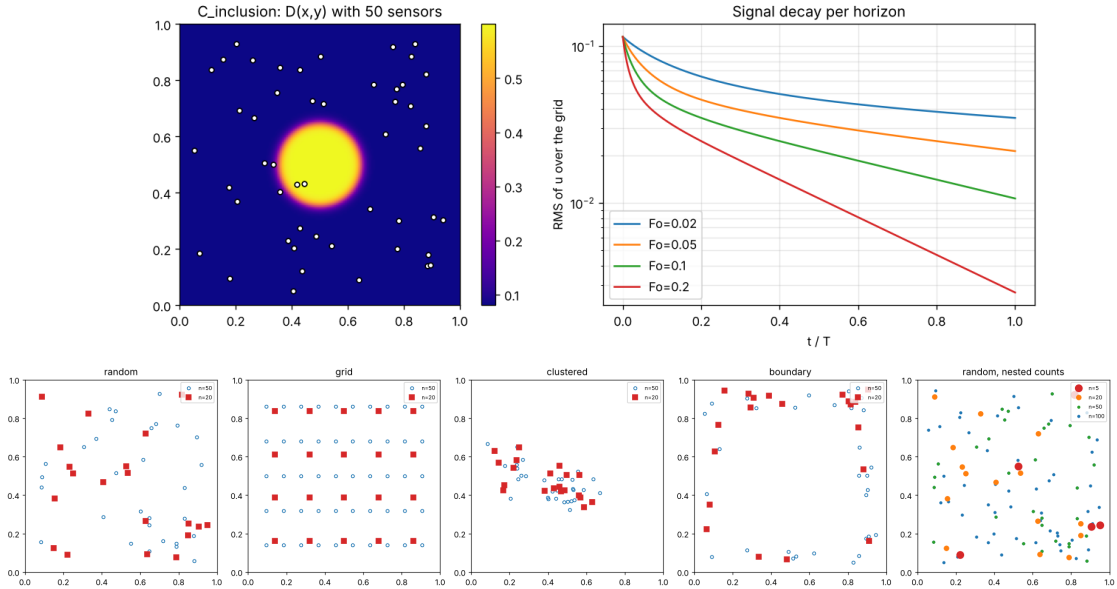


Figure 2: Top: inclusion family with 50 random sensors, and how fast the signal decays for each horizon. Bottom: the four sensor layouts.

6 Experiments

Metrics. (1) relL2(D) $= \|\hat{D} - D\|_2 / \|D\|_2$ on the 65^2 grid, the main ranking metric (lower is better). (2) Replay RMS/σ: $\hat{D}$ is run through the Crank–Nicolson solver and compared with the readings. About 1 means the data are explained to the noise level; much more than 1 means under-fitting, much less means over-fitting. This metric needs no ground truth, so it is the one shown to a user. (3) The best-constant baseline: the relL2(D) of the best single constant D. A method is useful only if it beats it.

Code. All code, trained models and results are in our GitHub repository [13], whose README maps every deliverable to its files:
`https://github.com/bloodrepaer/IE643-Binary-Brains-PINN-Inversion`

Hardware. The classical runs, the PINN benchmark and the 20/44-dataset studies ran on CPUs: a lap-

top, and 2-core cloud Linux containers with one thread per run. The neural campaign ran on a Windows laptop CPU, Kaggle notebooks (CPU, and GPU for some pilots) and Google Colab (T4 GPU). Wall times are therefore descriptive.

We list every experiment, including those with negative results.

1. **Verification.** Integrator order of accuracy against an exact-in-time matrix-exponential solution. Adjoint transpose identity, Taylor tests and finite-difference gradient checks (`test_adjoint.py`). Problem-file sufficiency: replaying the true D from the problem files alone.
2. **Classical inversion on all 389 datasets** (Tikhonov; TV also run on the inclusion and defect families).
3. **PINN loss-weight sweep**: 10 settings on the Gaussian bump (50 sensors, 5% noise).
4. **20-dataset head-to-head study**: representative datasets across all families; identical data and identical scoring.
5. **Sensor × noise × layout sweep, both methods**: 44 datasets on the inclusion (C) and bump (B); 5/20/50/100 random sensors × 0/1/5/10% noise, plus grid/clustered/boundary layouts at 20 and 50 sensors.
6. **Ablations**: 6,000 instead of 1,500 PINN epochs; no edge taper on D.
7. **Solver-bias test**: data regenerated with RK4, Crank–Nicolson, forward Euler, and with the adjoint's own solver and grid (a deliberate inverse crime); both methods re-run.
8. **Hypothesis suites "where should a PINN win?"** (81 paired datasets): H1 very sparse sensors (3/5/8); H2 unknown boundary condition (heated, ramped or pulsed edge, not given to either method); H3 unknown internal heat source; H4 heavy noise (10–50%); H5 long horizons; H6 more readings (100/400 sensors × 40 times); H7 unknown boundary with more readings; H9 features finer than the 16 × 16 control grid. Each test dataset has a control that differs only in the tested factor. For unknown physics both methods are told the same assumed model (zero boundary, no source).
9. **Neural-coefficient campaign**: neural adjoint on all 389 benchmark datasets plus the density and hypothesis collections; neural PINN on a canonical subset.
10. **Dense-sensor sweep**: 1,000/3,000/5,000/7,500 sensors × 1/5/10% noise on the smooth field (12 paired cells, 24 fits), plus a 10,000-sensor reference pair. An advantage counts only if the fit completed, has lower relL2(D), and passes a pre-declared replay screen ($\leq 1.1\sigma$ under both Crank–Nicolson and backward Euler) checked before the truth was opened.
11. **Re-runs of the reference papers**: independent re-implementations of the main tests of [17] and of [14].

7 Results

7.1 Solver and adjoint verification

Measured orders are 4.03 (RK4), 2.01 (Crank–Nicolson) and 1.00 (forward Euler) (Figure 3). Just above their stability limits, RK4 and forward Euler blow up, which is why the inversion uses an implicit scheme. The adjoint passes all checks: transpose identity to 4×10^{-16}, Taylor slopes 1.996–2.000, and adjoint vs

finite differences to 10^{-8}. Replaying the true D from the problem files alone gives median RMS/$\sigma = 1.01$ over 296 noisy datasets, so the problem files are sufficient and the noise model is right.

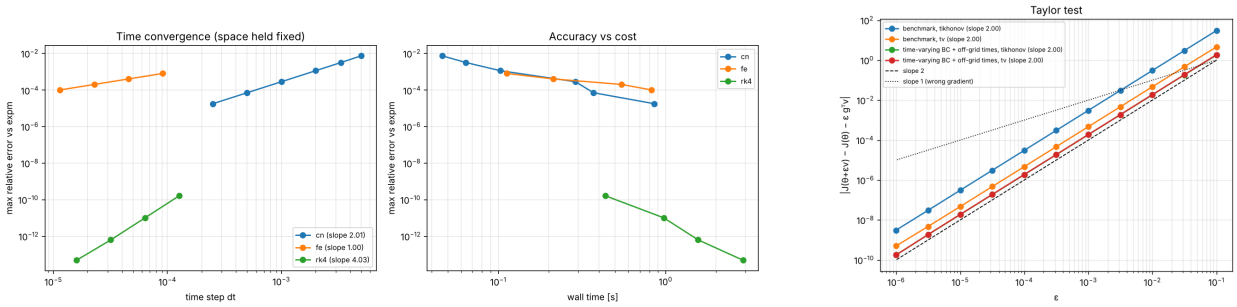


Figure 3: Left: convergence of the three integrators. Right: Taylor test of the adjoint gradient; the remainder falls with slope 2.

7.2 Classical inversion on the benchmark

The adjoint fits 324 of the 389 datasets to the noise level (median replay RMS/$\sigma = 0.98$). Its error falls with more sensors, more snapshots and less noise (Table 2). The inclusion is the hardest family: with 50 sensors it barely beats the best constant (0.66 vs 0.75), and full-field data bring it to 0.22. A negative result: on the layered field with $\leq 1\%$ noise, the discrepancy principle drives α very low, the method fits discretisation error at the sharp interfaces, and it overshoots (up to 1.40).

	Sensors (5% noise)					Noise (50 sensors)			
Family	5	20	50	100	full	0%	1%	5%	10%
A smooth	0.319	0.128	0.103	0.110	0.077	0.060	0.075	0.103	0.135
B bump	0.394	0.289	0.229	0.189	0.075	0.014	0.119	0.229	0.317
C inclusion	0.690	0.631	0.661	0.462	0.215	0.354	0.439	0.661	0.713
D layered	0.211	0.319	0.216	0.240	0.149	1.404	0.975	0.216	0.300
E random field	0.327	0.186	0.207	0.175	0.069	0.056	0.116	0.207	0.253
F defects	0.236	0.193	0.182	0.160	0.112	0.113	0.118	0.182	0.201

Table 2: Classical adjoint, relL2(D) (Tikhonov, Fo 0.1, 8 uniform times, random sensors).

7.3 PINN tuning

Over 10 loss-weight settings on the bump, the best relL2(D) was 0.398 (equal weights, no regulariser) and an over-fitting setting (data weight 250, PDE weight 2) gave 0.873, against 0.229 for the adjoint. No setting came within 1.7× of the adjoint. More importantly, the network's own data fit was $\chi^2 \approx 0.95$ for nine very different D estimates (0.40–0.65), while replaying those D through the solver gave χ^2 from 3 to 56. A PINN can fit the readings with the wrong coefficient. We therefore score every estimate by replaying it through the solver, never by the network's own fit.

7.4 Head-to-head on representative datasets

On 20 datasets the adjoint has the lower error on all 20, with a median ratio of 2.0× and median relL2(D) 0.23 vs 0.47 (Figure 4). It beats the best constant on 20/20, the PINN on 8/20. Median replay RMS/σ is

0.97 (adjoint) vs 3.7 (PINN), and median CPU time is 46 s vs 125 s. The gap is smallest where the problem is hardest for everyone (sharp inclusion, few sensors) and largest where data are rich (full field: 0.030 vs 0.405). The adjoint turns extra data into accuracy; the PINN does not, partly because its data term is a mean over readings.

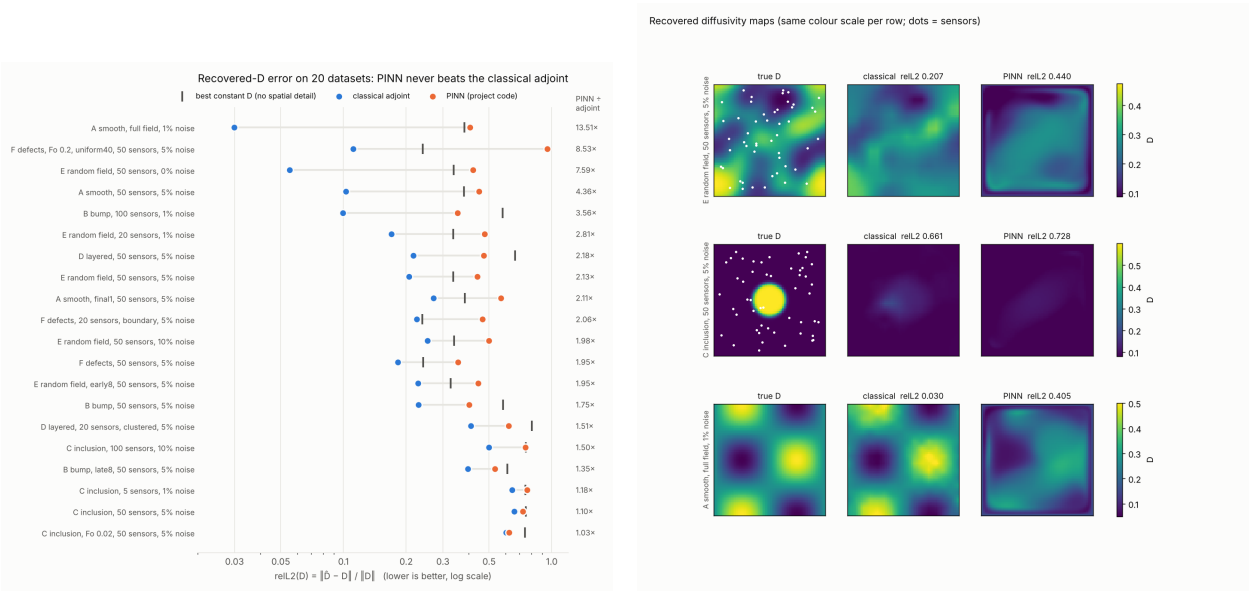


Figure 4: Left: relL2(D) on 20 datasets (log scale; bars are the best constant). Right: recovered maps for three of them.

Ablations and solver bias. Training the PINN for 6,000 epochs improved it (e.g. smooth field 0.447→0.343) but it still lost 4/4. Removing the edge taper made every case worse. Changing the data-generating solver moved the adjoint's error by at most 0.004, even under the inverse crime. The PINN's error moved only by its run-to-run variation (±0.03). The adjoint's advantage is therefore not an artefact of sharing a solver with the data.

7.5 Sensor density, layout and noise (both methods)

In the 44-dataset sweep the adjoint has the lower error on 41 (median relL2(D) 0.36 vs 0.61; best constant 0.68), see Figure 5. On the bump both methods improve with more sensors and less noise, and the adjoint is 1.2–24× better. On the inclusion the PINN stays at 0.68–0.76, within 10% of the best constant, at every sensor count and noise level. The adjoint reaches 0.19–0.25 with 100 sensors at 1–5% noise but is erratic below that. A grid layout beats random sensors on the bump for both methods (adjoint 0.116 vs 0.229 at 50 sensors), and boundary-only sensors are worst. The PINN's three wins are all on the inclusion with too little information: 5 random sensors at 5% noise (0.746 vs 0.779), 20 boundary sensors (0.766 vs 1.529) and 50 boundary sensors (0.740 vs 0.747). There the PINN stays at the constant level while the adjoint fits noise and invents structure. This is graceful degradation, not a better reconstruction.

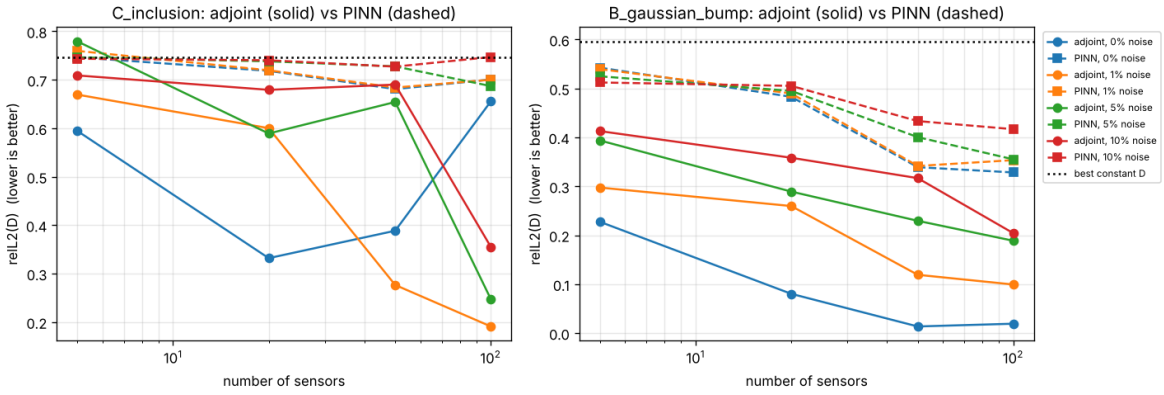


Figure 5: relL2(D) against sensor count at four noise levels; adjoint solid, PINN dashed, best constant dotted.

7.6 Where the PINN is better: hypothesis suites

Table 3 and Figure 6 summarise the 81 paired hypothesis datasets. With the physics fully known (sparse sensors, heavy noise, long horizons, more data), the adjoint wins 58 of 59. With an *unknown boundary condition or source*, the PINN wins 21 of 22. Figure 7 shows why. Once the boundary is wrong, even the *true* D replays at about 9–21σ under the assumed model, so an exact-constraint method must distort D to explain the data (median relL2(D) 5.0). The PINN's soft PDE penalty, and in H2/H7 its free walls, absorb the mismatch, and it stays below the best constant (0.38 vs 0.58). This is a win over the standard adjoint formulation. A classical method that also inferred the boundary values or the source would be a stronger comparator, and we list it as future work.

Group	Datasets	PINN wins	Adjoint	PINN	Best const.
Physics known (H1, H4–H6, H9, controls)	59	1	0.123	0.412	0.344
BC or source unknown (H2u, H3u, H7u)	22	21	5.011	0.379	0.580

Table 3: Hypothesis suites: number of PINN wins and median relL2(D). "Unknown" means withheld from both methods.

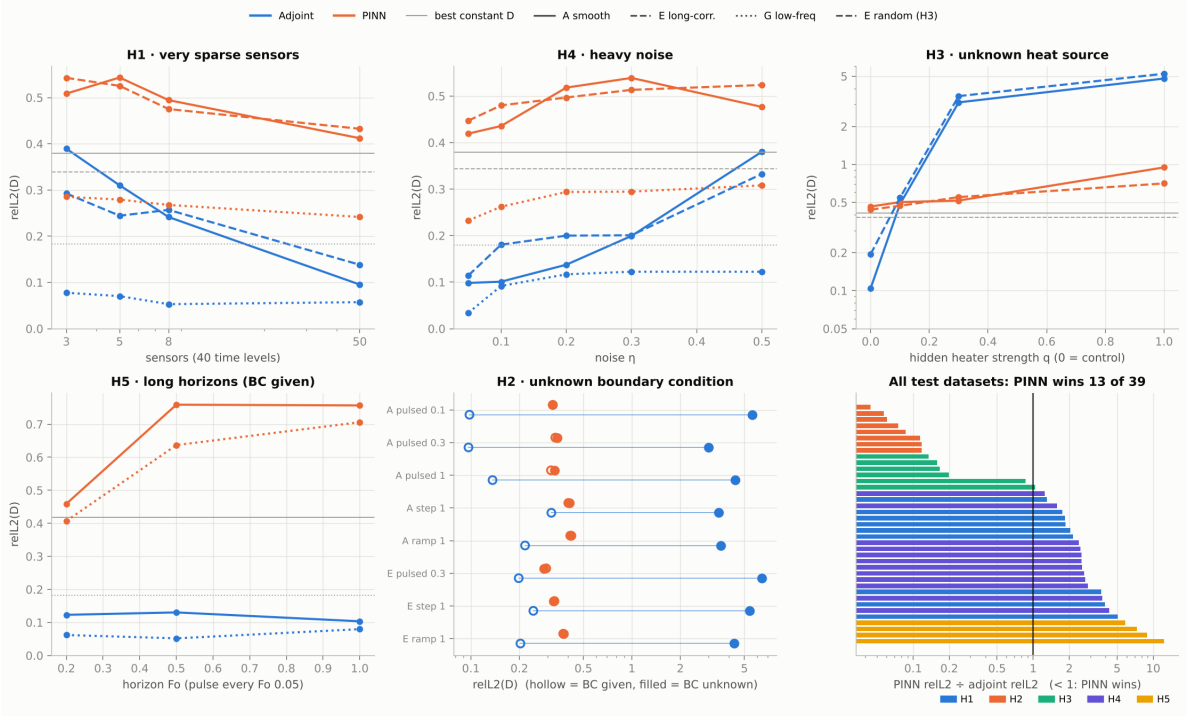


Figure 6: Hypothesis suites H1–H5. The PINN wins only where the boundary condition (H2) or heat source (H3) is unknown.

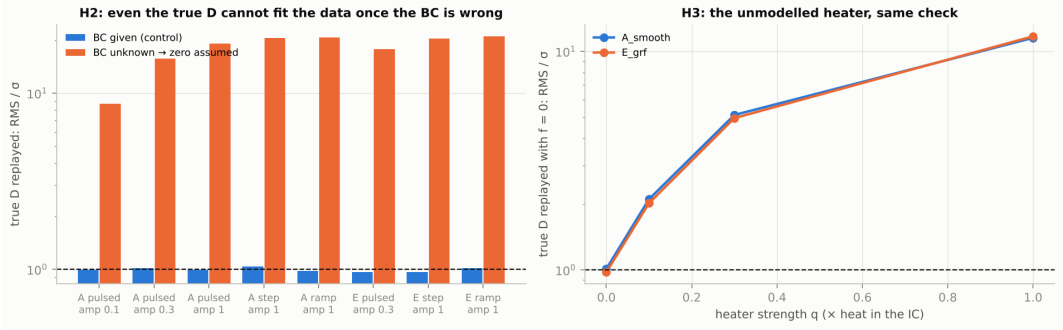


Figure 7: Why the adjoint fails under unknown physics: even the true D cannot fit the data under the assumed boundary (left) or source (right).

7.7 Neural coefficient and dense sensors

Giving the adjoint the same neural representation of D did not change its accuracy: the neural adjoint and the grid adjoint have median relL2(D) 0.226 vs 0.225 on the 389 benchmark datasets. With up to a few hundred sensors the neural PINN behaves like the original PINN. On canonical cases it fits the sensors but its D replays at about 5.8σ (median relL2(D) 0.61). The picture changes with very dense data (Table 4, Figure 8). At 5% noise the PINN's error falls from 35.4% (1,000 sensors) to 8.6% (7,500), while the matched neural adjoint stays at 21–25%. At 7,500 sensors both pass both replay screens, so this is a

consistency-qualified PINN advantage. A 10,000-sensor reference pair gives 6.8% vs 26.4%. At 1% noise the PINN has lower error at every density but fails the replay screens. At 10% noise the adjoint is better up to 5,000 sensors; at 7,500 the PINN's error is slightly lower (22.6% vs 23.3%) but it fails the backward-Euler replay screen. These are single-field, single-seed results with capped optimisation, so they establish a regime worth studying, not a general rule.

Sensors	Noise	PINN relL2(D)	Adjoint relL2(D)	PINN replay CN / BE (σ)	Adjoint replay CN / BE (σ)
1,000	5%	35.37%	21.45%	1.924 / 2.005	1.076 / 0.967
3,000	5%	16.48%	23.01%	1.136 / 1.150	1.095 / 0.992
5,000	5%	13.03%	23.79%	1.102 / 1.068	1.098 / 0.992
7,500	5%	**8.62%**	25.09%	1.097 / 1.025	1.098 / 0.994
7,500	1%	5.61%	45.64%	2.969 / 1.873	2.595 / 1.068
7,500	10%	22.64%	23.26%	1.098 / 1.122	1.019 / 0.993

Table 4: Dense-sensor sweep (smooth field, temporal neural PINN vs backward-Euler neural adjoint). Replay is checked against 1.1σ.

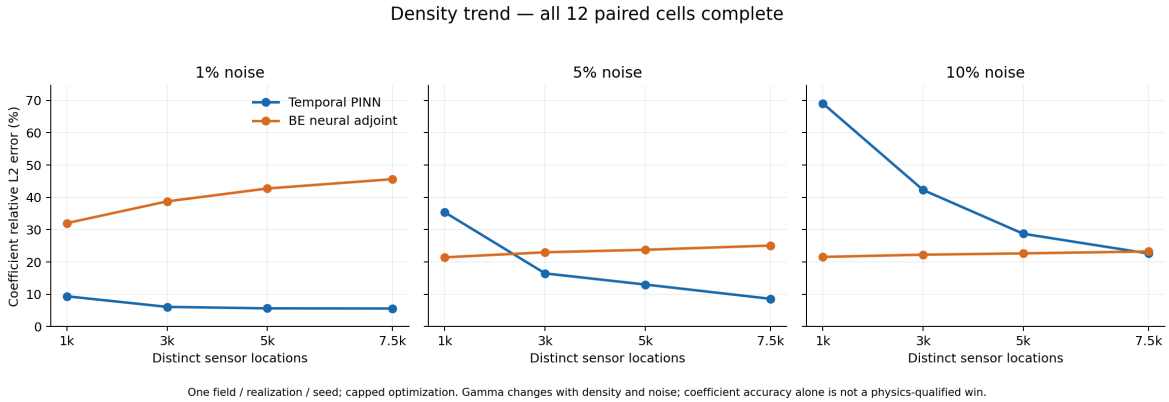


Figure 8: Coefficient error against sensor density at three noise levels (12 paired cells).

7.8 Reference-paper re-runs

Our re-implementation of [17] reproduces their Burgers and Darcy numbers (e.g. Burgers grid adjoint 1.55×10^{-3} vs reported 1.549×10^{-3}; Darcy at 1% noise, adjoint 0.248, PINN 0.254), including the result that a PINN beats the adjoint when the unknown is a network at a small iteration budget. It also reproduces the cost trend on 3D Allen–Cahn. On the four cases of [14], a 10-second classical finite-difference least-squares fit (0.8–3.3% error) and our adjoint (2.7–6.3%) match or beat the reported PINN (4.6–6.3%). We also found that their backward-Euler loss is biased unless Δt is much smaller than the data spacing: with a perfect concentration field it gives 31% error at $\Delta t = 0.01$. Their sparse-data results (6.45% at 250 points) were better than our adjoint on comparable data (8.7%). That was the one PINN advantage we could not rule out from the papers, and it is consistent with our own dense-data finding.

7.9 Interface

Figure 9 shows the results tab for the inclusion dataset. Classical: relL2(D) 0.661, replay 0.98σ. PINN: 0.730, 2.38σ. Best constant: 0.751. The bundled models reproduce the stored comparison numbers when

rescored in the app.

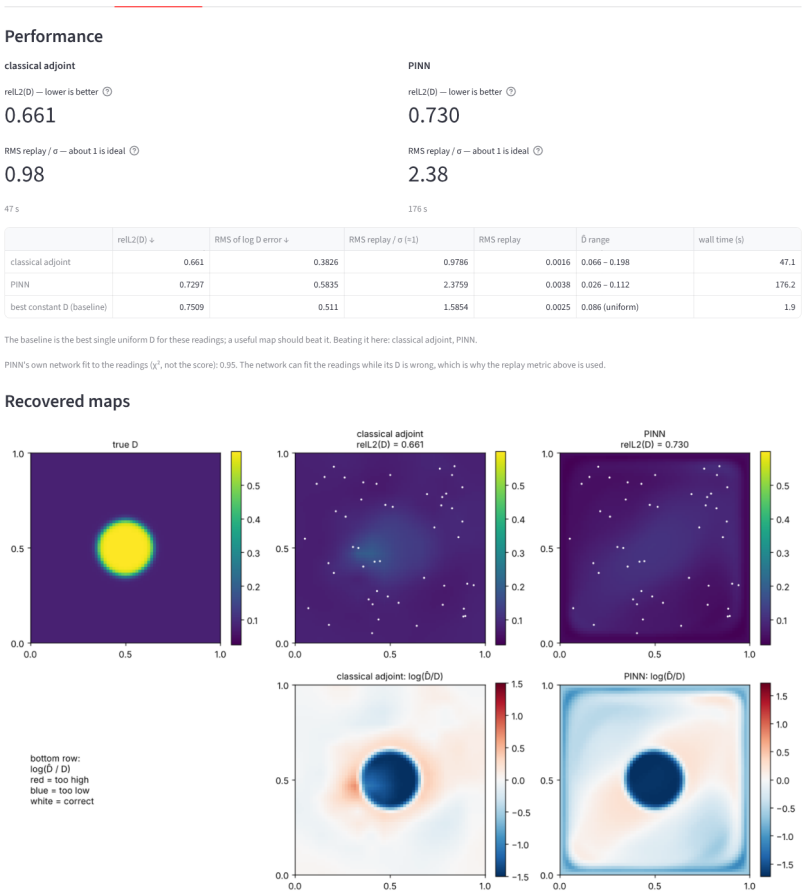


Figure 9: Interface results tab: displayed metrics, recovered maps and $\log(\hat{D}/D)$ error maps.

8 Plan for Novelty Assessment

1. **A fair classical comparator under unknown physics.** Add the boundary values (a few parameters per edge and time) and a low-dimensional source as extra unknowns in the adjoint, and re-run H2, H3 and H7. This tests whether the PINN's 21/22 advantage survives against a stronger baseline, or whether it is due to the standard formulation's rigidity.

2. **Hybrid PINN→adjoint.** Use the PINN's D (robust under unknown physics and dense data) to warm-start the adjoint, and measure total cost honestly, including the PINN's training time.

3. **Make the PINN use more data.** Replace the mean data loss by a σ-weighted sum, train in float64 with a quasi-Newton optimiser, and repeat the dense-sensor sweep over several fields and seeds to test whether the density advantage is general.

4. **Uncertain initial conditions**, known only at the sensors or with 5–10% error. This is another regime where an exact-constraint method should suffer.

5. **Uncertainty quantification** for $\hat{D}$ from a Gauss–Newton Hessian (adjoint) or an ensemble of PINNs.

9 Conclusion

We built the full pipeline for recovering a spatially varying diffusivity from sparse, noisy heat data. It consists of a verified finite-volume forward solver and a 389-dataset benchmark, a classical adjoint inversion with exact gradients, a PINN, a neural-coefficient extension in which both methods share the same network for D, and an interface that runs both methods on generated, benchmark or uploaded data. In the standard setting (exact initial and boundary conditions, up to a few hundred sensors) the classical method is about twice as accurate, fits the data to the noise level, and is faster. Representing D by a network does not change this. The PINN is the better choice in three regimes that we identified with matched controls: when the boundary condition or a heat source is unknown, when the data are very dense, and when the data are too sparse to identify D, where it degrades gracefully instead of overshooting. Its value is flexibility and robustness to imperfect physics rather than accuracy on a perfectly specified problem. Future work will test that advantage against a classical method that also infers the unknown physics, and combine the two methods.

References

[1] Richard H. Byrd, Peihuang Lu, Jorge Nocedal, and Ciyou Zhu. A limited memory algorithm for bound constrained optimization. *SIAM Journal on Scientific Computing*, 16(5):1190–1208, 1995.

[2] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015. `https://arxiv.org/abs/1412.6980`.

[3] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. `https://arxiv.org/abs/1711.05101`.

[4] V. A. Morozov. On the solution of functional equations by the method of regularization. *Soviet Mathematics Doklady*, 7:414–417, 1966.

[5] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, 2019.

[6] R.-E. Plessix. A review of the adjoint-state method for computing the gradient of a functional with geophysical applications. *Geophysical Journal International*, 167(2):495–503, 2006.

[7] Nasim Rahaman, Aristide Baratin, Devansh Arpit, Felix Draxler, Min Lin, Fred Hamprecht, Yoshua Bengio, and Aaron Courville. On the spectral bias of neural networks. In *International Conference on Machine Learning*, 2019. `https://arxiv.org/abs/1806.08734`.

[8] Maziar Raissi, Paris Perdikaris, and George Em Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019. `https://doi.org/10.1016/j.jcp.2018.10.045`.

[9] Rolf Rannacher. Finite element solution of diffusion problems with irregular data. *Numerische Mathematik*, 43:309–327, 1984.

[10] Leonid I. Rudin, Stanley Osher, and Emad Fatemi. Nonlinear total variation based noise removal algorithms. *Physica D: Nonlinear Phenomena*, 60:259–268, 1992.

[11] Streamlit Inc. Streamlit: a faster way to build and share data apps. `https://streamlit.io`, 2026. Accessed on: 20th October, 2026.

[12] Matthew Tancik, Pratul P. Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan T. Barron, and Ren Ng. Fourier features let networks learn high frequency functions in low dimensional domains. In *Advances in Neural Information Processing Systems*, 2020. https://arxiv.org/abs/2006.10739.

[13] Jash Taparia and Mihir Gune. IE 643 project code: PINN vs adjoint inversion of a spatially varying diffusivity. https://github.com/bloodrepaer/IE643-Binary-Brains-PINN-Inversion, 2026. Accessed on: 24th October, 2026.

[14] S. Thakur, E. Esmaili, S. Libring, L. Solorio, and A. M. Ardekani. Inverse resolution of spatially varying diffusion coefficient using physics-informed neural networks. *arXiv preprint arXiv:2403.03970*, 2024. https://arxiv.org/abs/2403.03970.

[15] Curtis R. Vogel. *Computational Methods for Inverse Problems*. SIAM, 2002.

[16] Sifan Wang, Yujun Teng, and Paris Perdikaris. Understanding and mitigating gradient flow pathologies in physics-informed neural networks. *SIAM Journal on Scientific Computing*, 43(5):A3055–A3081, 2021. https://arxiv.org/abs/2001.04536.

[17] Z. Zhang, A. Alla, and G. E. Karniadakis. Adjoint method versus physics-informed neural networks in PDE-constrained inverse problems. *arXiv preprint arXiv:2606.12337*, 2026. https://arxiv.org/abs/2606.12337.